Voltera: PV Detection from Electricity Consumption

Finding solar installations in the patterns of everyday demand

Alessandro Tognoni Jeffre Daniel Martinez Marroquin Tarun Deepak

1 Time-series problem and data

We built Voltera to answer three questions from a customer's electricity history: is there a photovoltaic (PV) installation, when did it first come into service, and what is its nominal capacity? The AIL challenge gave us 15-minute grid-withdrawal measurements, 40 labelled development customers and 1,148 additional unlabelled histories. Another 20 customers were reserved for hidden testing [1]. We could use the timestamps, but not weather, irradiation or location data.

Our starting point was the effect of solar generation on the meter. Once panels supply part of a building's daytime demand, it needs less electricity from the grid. Depending on the meter, surplus production can appear as negative readings or simply as zero imports. Neither gives us a separate measurement of generation. We therefore looked for a change in the shape of demand, rather than treating a low midday reading as proof of PV.

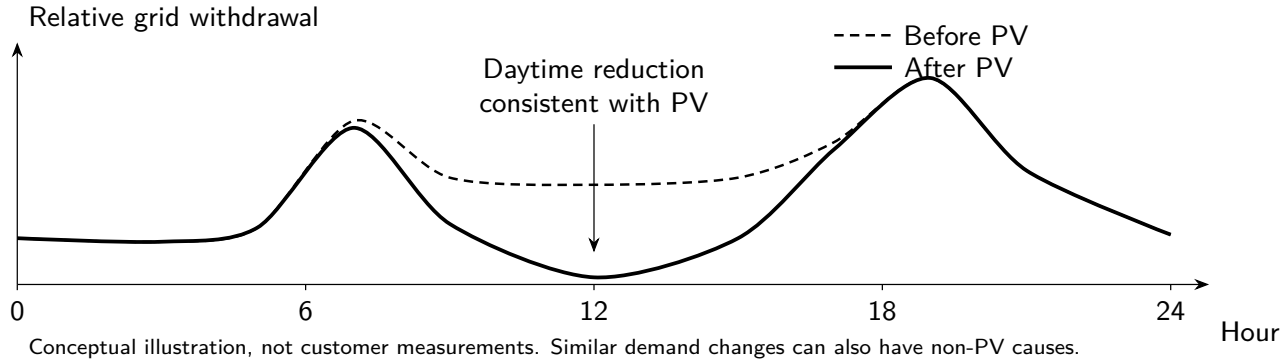


Figure 1: We looked for a persistent daytime reduction while using the rest of the day as a reference. These curves illustrate the idea, not a measured customer or a prediction.

That distinction matters because ordinary consumption already has considerable structure. Morning and evening routines create daily peaks, working days differ from weekends, and demand changes with the seasons. A springtime reduction can resemble a new solar installation. Conversely, a system installed in winter may not leave a strong daytime signature until later. Holidays, occupancy changes and batteries make the interpretation harder still. We wanted to find a lasting, solar-like change against this background, rather than forecast the next reading.

Before modelling, we built a common preparation pipeline to validate the input schema, order readings by epoch timestamp and handle duplicates. We kept missing readings separate from measured zeros, since filling a midday gap with zeros would create exactly the signal we were trying to detect. Epoch time also helped us resolve daylight-saving transitions. Our canonical local-day summaries account for 92, 96 or 100 intervals, while the physical detector uses a separate fixed UTC+1 daily matrix with 96 slots. Coverage checks keep sparse days and weeks from dominating the evidence.

We also kept the roles of the datasets separate. The unlabelled histories could help us learn patterns, but an absent label did not make a customer PV-negative. Synthetic customers let us test installations, gaps and demand changes under known conditions. We distinguish these diagnostics from aggregate development evaluations and include no real-customer traces.

2 Time-series methodology

We combined two views of the same history. One describes how daytime demand changes relative to the customer's other consumption. The other learns whether weekly profiles resemble those of PV customers. We kept the date and capacity estimates tied to explicit temporal and physical features, rather than asking one model to produce all three answers.

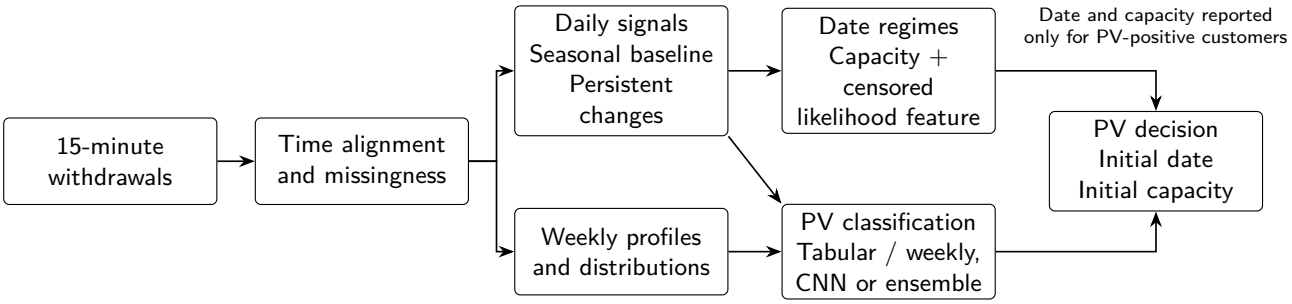


Figure 2: We use daily signals for physical evidence and weekly profiles for learned presence detection. Date and capacity estimates are reported only for customers classified as having PV.

For the physical view, we summarized midday, night and evening consumption, their relative levels, the share of zero readings and the duration of the zero-import window around noon. Night and evening demand gave us useful reference signals: a reduction across the whole day calls for a different interpretation from a change concentrated around midday. We also retained coverage and export features where the readings supported them.

We then removed typical seasonal variation from the midday/night log-ratio using a week-of-year baseline fitted on non-PV training customers. Binary segmentation proposed change points on seven-day summaries, followed by local refinement. We checked whether a drop persisted and whether similar changes recurred in other years. For dating, the current implementation favors the earliest sufficiently supported onset, while classification retains its standard candidate features. These checks help distinguish a new installation from a recurring seasonal pattern, although they cannot resolve every demand change.

For presence detection, we implemented logistic regression, shallow CatBoost and weekly-evidence models. We also adapted the CNN architecture of Neubert et al. [2], combining a 168-hour profile with a 100-point consumption-density representation. We normalized both inputs per customer, trained the branches separately and fitted a combined head with frozen branches. Window weights give each customer equal total contribution. We summarize CNN evidence using the weakest probability in the strongest run of four consecutive weeks. This preserves recent evidence but can miss very short histories. We also added an ensemble that averages the CNN probability with the selected tabular/weekly classifier. Shared inner folds select its threshold and compare it with the other candidates; enabling the CNN does not force its adoption.

For dates, we distinguish a visible event, PV present at the start, and insufficient evidence, with training-only calibration. For capacity, we combine midday changes, load levels and zero-window width with Ridge or CatBoost regression of log-capacity and a fitted scale factor. We later added a Tobit-style censored-likelihood estimate: a zero-import reading tells us that generation covered demand, not that generation was zero. We model pre-onset load per time slot as lognormal, adjust it using night-load changes, and combine a timestamp-only solar shape with the training-fitted seasonal curve. Positive imports contribute a likelihood density; near-zero readings contribute a probability bound. The resulting log-capacity estimate is an additional regression feature, not a direct production measurement. It requires sufficient pre-onset and April–August post-onset coverage and stays missing otherwise. Mainline now includes it by default, with an explicit opt-out.

This follows the censored-observation idea of Brown et al. [3], but our implementation uses lognormal load and timestamp-only solar proxies rather than reproducing their full model.

Separately, we explored extensions using persistent increases in season-normalized deficits and year-over-year comparisons. This experiment reports total capacity after each event and leaves the initial predictions unchanged. It is not integrated into main inference, which still writes a header-only extension file. We implemented the work in Python using NumPy/pandas, SciPy, scikit-learn, CatBoost and PyTorch.

3 Experimental results and discussion

With only 40 labelled customers, we needed to be careful about what counted as independent evidence. We split by customer, keeping each customer's windows together. Inner folds select configurations and thresholds; pooled outer predictions assess the procedure. We used shared splits for comparisons, following reproducible time-series benchmarking practice [4]. We also logged design changes informed by outer results. An audit later included lock-box customers, so we cannot describe subsequent evidence as a clean independent lock-box evaluation.

The challenge score is $S = F_1(70 + 15D + 15P)$. Date reward falls to zero at 30 days of error, and capacity reward at 50% absolute percentage error. Both are averaged only over correctly detected PV customers. We therefore consider detection and conditional errors together: changing which customers are detected can change these errors without changing the estimators. The results below are development comparisons, not hidden-test scores. Repeated folds and bootstrap intervals do not create additional independent customers or capture all selection uncertainty.

Our team's decision log records adoption of the CNN probability ensemble after a paired development comparison gave a score increase of 4.06 points, with a 90% interval of [1.40, 7.56]. Pseudo-labelling was not selected in any of the 20 outer fits in its comparison. A subsequent pre-registered capacity comparison used three fresh customer-split seeds on all 40 development customers. Adding the fixed censored-likelihood feature raised capacity reward from 0.419 to 0.537, with a paired total-score increase of 1.44 points and interval [0.45, 2.42]; detection and dates were identical. These are recorded team evaluations, not runs repeated for this report. Fresh splits still reuse the development customers, so we do not treat them as a new independent dataset.

Our separate synthetic extension experiment used nine scenarios with seven true events. Physical detection found three events with two false predictions; a pretrained filter removed one false prediction but found no extra events. The two-event case recovered only its second extension, and short tails and clipping remained difficult. This informed our research, but not a claim of real extension accuracy or an integrated submission feature.

The Transformer comparison grew out of a broader idea we explored during the hackathon. We wanted to teach PatchTST the "language" of electricity time series [5], using four-hour patches as words and a week as their context. By hiding readings and reconstructing them, we could learn from unlabelled histories before applying PV labels. We trained small and medium encoders, then explored frozen features, limited fine-tuning and a CNN-gated hybrid. The difficulty was turning reconstruction into the answers we needed. Reconstructed-minus-observed demand was not established as PV generation, and learning typical consumption did not directly teach installation dates or nominal capacity. We had not shown enough reliable downstream benefit to justify the extra complexity. We therefore moved toward task-specific CNN detection and physical/statistical estimation. PatchTST is not in the chosen Voltera pipeline; its extension filter remains a separate experiment.

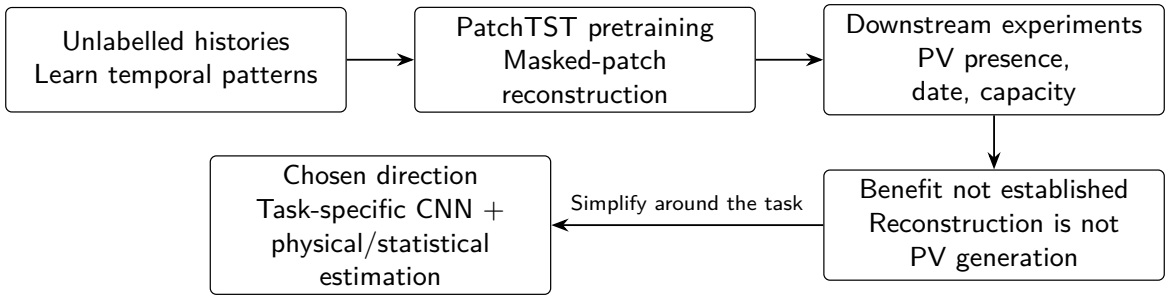


Figure 3: Our Transformer work taught us to distinguish learning consumption patterns from answering the PV tasks. We retained the trained encoders for research but did not include PatchTST in the chosen pipeline.

Alongside the models, we built offline inference, exact CSV outputs, serialization, hygiene checks and a local GUI. We used LLM coding assistants for implementation, tests, debugging and drafting, but no LLM runs inside training or inference. We remain responsible for checking the results and reviewing this report before submission.

References

[1] AIL SA, "AI-based PV Detection from Electricity Consumption Profiles."

[2] M. Neubert, O. Gnepper, O. Mey, and A. Schneider, "Detection of Electric Vehicles and Photovoltaic Systems in Smart Meter Data," *Energies*, vol. 15, no. 13, p. 4922, 2022, doi: 10.3390/en15134922.

[3] J. Brown, A. Abate, and A. Rogers, "Disaggregation of household solar energy generation using censored smart meter data," *Energy and Buildings*, vol. 231, p. 110617, 2021, doi: 10.1016/j.enbuild.2020.110617.

[4] aeon developers, "aeon-benchmark: Reproducible time-series machine-learning experiments." [Online]. Available: https://github.com/aeon-toolkit/aeon-benchmark

[5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *ICLR*, 2023. [Online]. Available: https://arxiv.org/abs/2211.14730